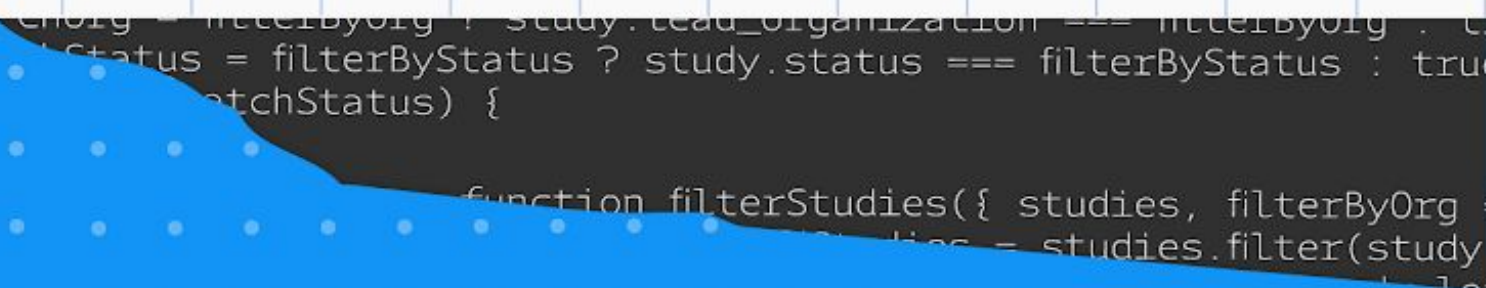




Synchronizing UI and database operations



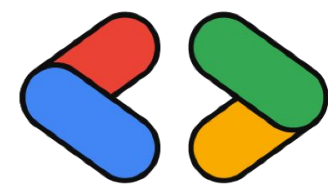
CRUD in Database Systems

Most applications require CRUD operations

CRUD stands for:

- Create → add new student
- Read → view students
- Update → edit student data
- Delete → remove student

Today we implement full CRUD.



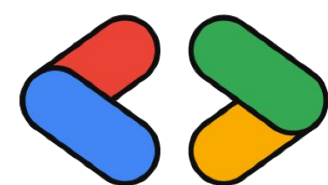
Create Data in Supabase

Example: inserting a new student

```
const { data, error } = await supabase
  .from("students")
  .insert([
    {
      name: "Rina Putri",
      major: "Computer Science",
      year: 2024
    }
  ])
```

Equivalent SQL operation:

```
INSERT INTO students (name, major, year)
VALUES ('Rina Putri', 'Computer Science', 2024);
```



Google Developer Group
Universitas Sriwijaya

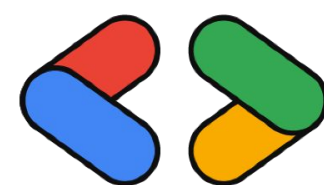
Update Data in Supabase

Example: inserting a new student

```
const { data, error } = await supabase
  .from("students")
  .update({ major: "Software Engineering" })
  .eq("id", 1)
```

Equivalent SQL operation:

```
UPDATE students
SET major = 'Software Engineering'
WHERE id = 1;
```



Delete Data in Supabase

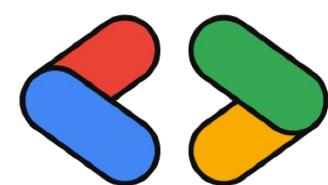
Example: deleting a student.

```
const { error } = await supabase
  .from("students")
  .delete()
  .eq("id", 1)
```

Equivalent SQL:

```
DELETE FROM students
WHERE id = 1;
```

These operations modify server data.



Why Mutations Are Needed

In the previous session we used queries to read data.

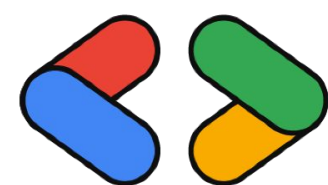
`useQuery()`

But queries should not modify server data.

TanStack Query provides mutations for operations that change data

Queries → Read data

Mutations → Create / Update / Delete



Creating a Mutation

Example: creating a student.

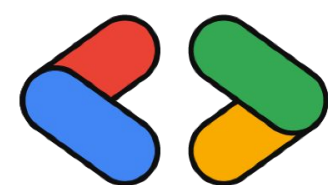
```
const mutation = useMutation({  
  mutationFn: createStudent  
})
```

Trigger mutation:

```
mutation.mutate(newStudent)
```

TanStack Query handles:

- loading state
- succes state
- error state



Google Developer Group
Universitas Sriwijaya

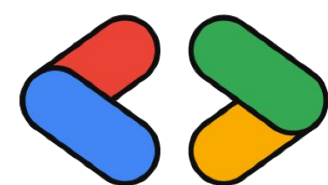
Syncing UI with Database

After creating or updating data, the UI must update.

TanStack Query provides query invalidation.

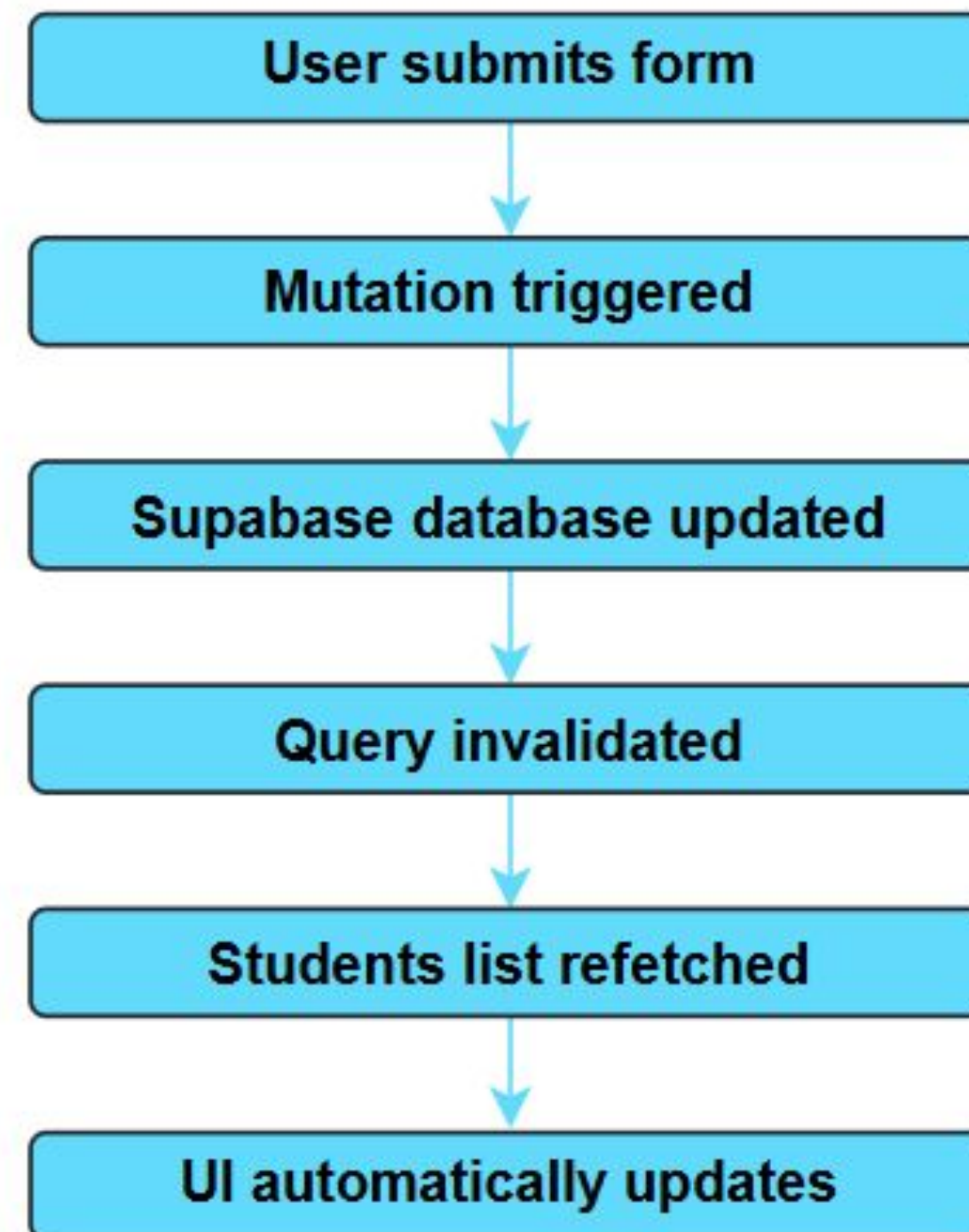
```
queryClient.invalidateQueries({  
  queryKey: ["students"]  
})
```

This tells TanStack Query to refetch updated data.

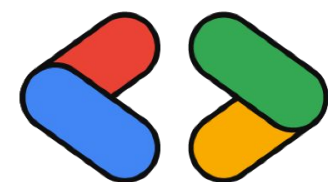


Example Flow

After creating or updating data, the UI must update



No manual refetching required





Thank You !